学号：2106410119

智能手机应用程序开发作业

2023 年_秋_季学期

学　　院：计算机科学与工程学院

专业班级：计算机 210　班

姓　　名：陈芷涵

第一章作业：

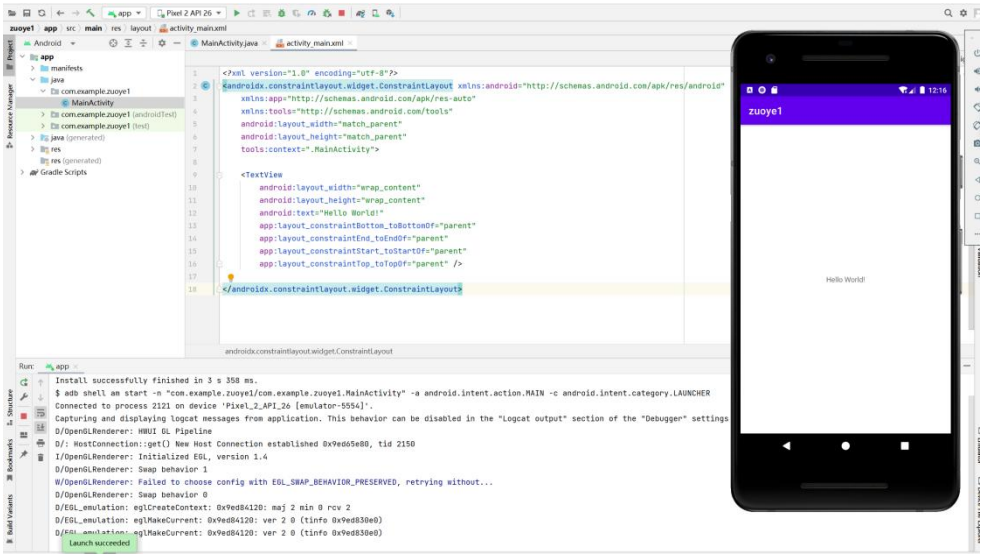
P22 1．下载最新版本的 Android Studio，安装后如果需要更新 Gradle，则将其更新到最新版本。

P22 2．创建一个任意类型的手机 App 工程，分别使用虚拟设备和物理设备运行。

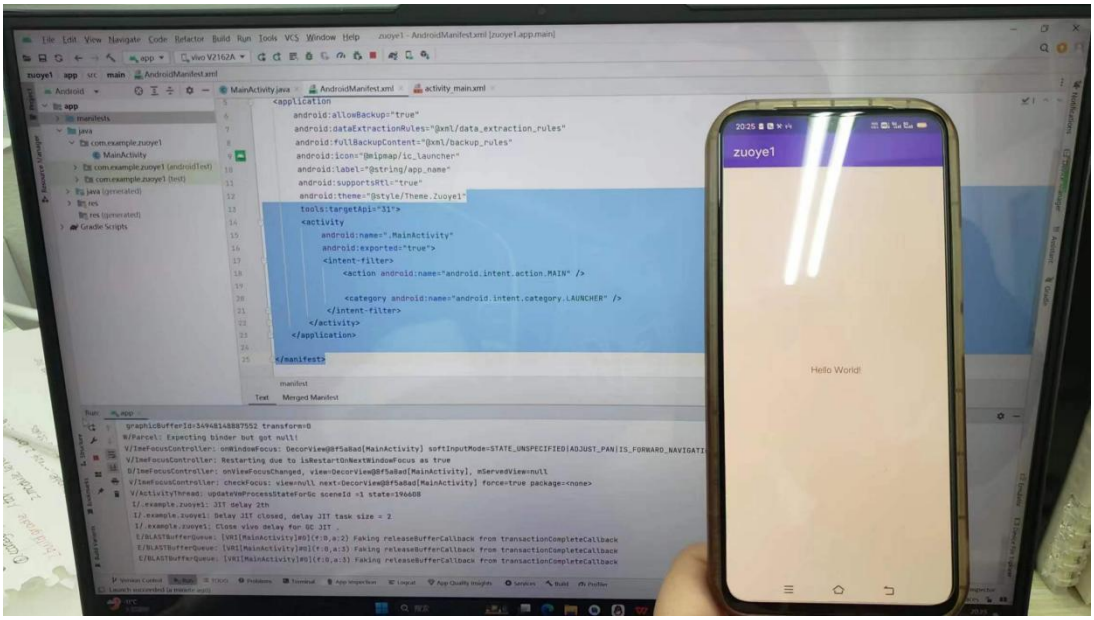
关键代码：

```
<application
    android:allowBackup="true"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:supportsRtl="true"
    android:theme="@style/Theme.Zuoye1"
    tools:targetApi="31">
    <activity
        android:name=".MainActivity"
        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</application>
```

虚拟设备：



物理设备：



第二章作业：

P73　2.2　实现多选题的界面效果。

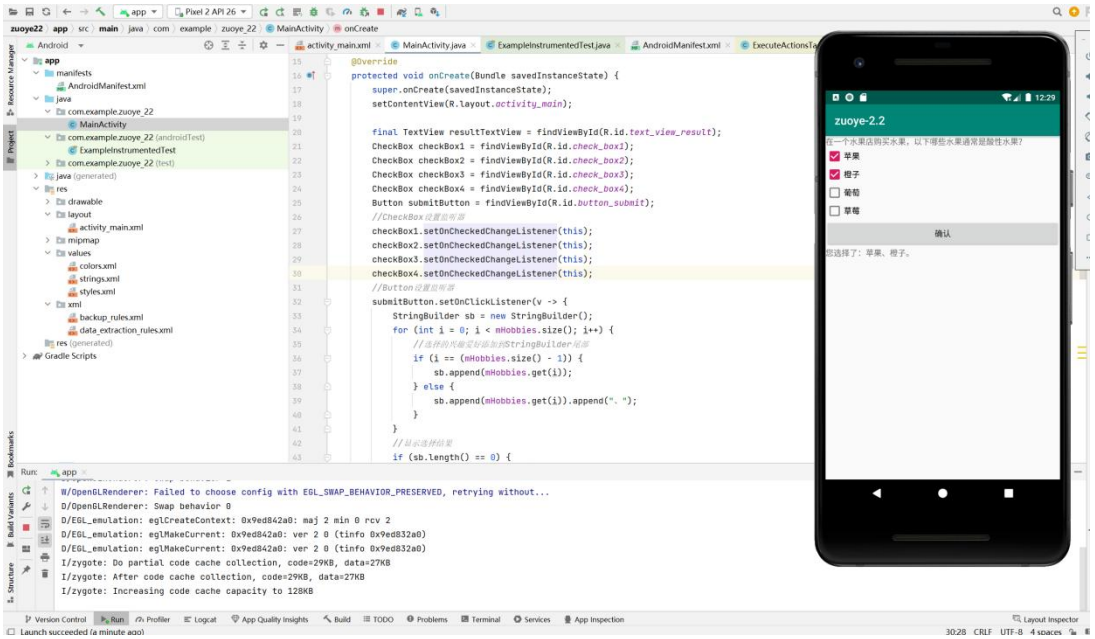
代码：

```
package com.example.zuoye_22;
import android.annotation.SuppressLint;
import android.os.Bundle;
import android.widget.Button;
import android.widget.CheckBox;
import android.widget.CompoundButton;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;
import java.util.ArrayList;

public class MainActivity extends AppCompatActivity implements CompoundButton.OnCheckedChangeListener {
    private final ArrayList<String> mHobbies = new ArrayList<>();
    @SuppressLint("SetTextI18n")
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
```

```
        final TextView resultTextView = findViewById(R.id.text_view_result);
        CheckBox checkBox1 = findViewById(R.id.check_box1);
        CheckBox checkBox2 = findViewById(R.id.check_box2);
        CheckBox checkBox3 = findViewById(R.id.check_box3);
        CheckBox checkBox4 = findViewById(R.id.check_box4);
        Button submitButton = findViewById(R.id.button_submit);
        //CheckBox 设置监听器
        checkBox1.setOnCheckedChangeListener(this);
        checkBox2.setOnCheckedChangeListener(this);
        checkBox3.setOnCheckedChangeListener(this);
        checkBox4.setOnCheckedChangeListener(this);
        //Button 设置监听器
        submitButton.setOnClickListener(v -> {
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < mHobbies.size(); i++) {
                //选择的兴趣爱好添加到 StringBuilder 尾部
                if (i == (mHobbies.size() - 1)) {
                    sb.append(mHobbies.get(i));
                } else {
                    sb.append(mHobbies.get(i)).append("、");
                }
            }
            //显示选择结果
            if (sb.length() == 0) {
                resultTextView.setText("您还没有进行选择了！");
            } else {
                resultTextView.setText("您选择了：" + sb + "。");
            }
        });
    }
    //提示 '@param compoundButton' tag description is missing， '@param isChecked' tag description is missing
    // 自动 fix
    /**
     * 实现改变选项的接口方法
     *
     * @param compoundButton
     * @param isChecked
     * @return void
```

```
        */
        @Override
        public void onCheckedChanged(CompoundButton compoundButton, boolean isChecked) {
            if (isChecked) {
                //添加到数组
                mHobbies.add(compoundButton.getText().toString().trim());
            } else {
                //从数组中移除
                mHobbies.remove(compoundButton.getText().toString().trim());
            }
        }
}
```

运行截图：



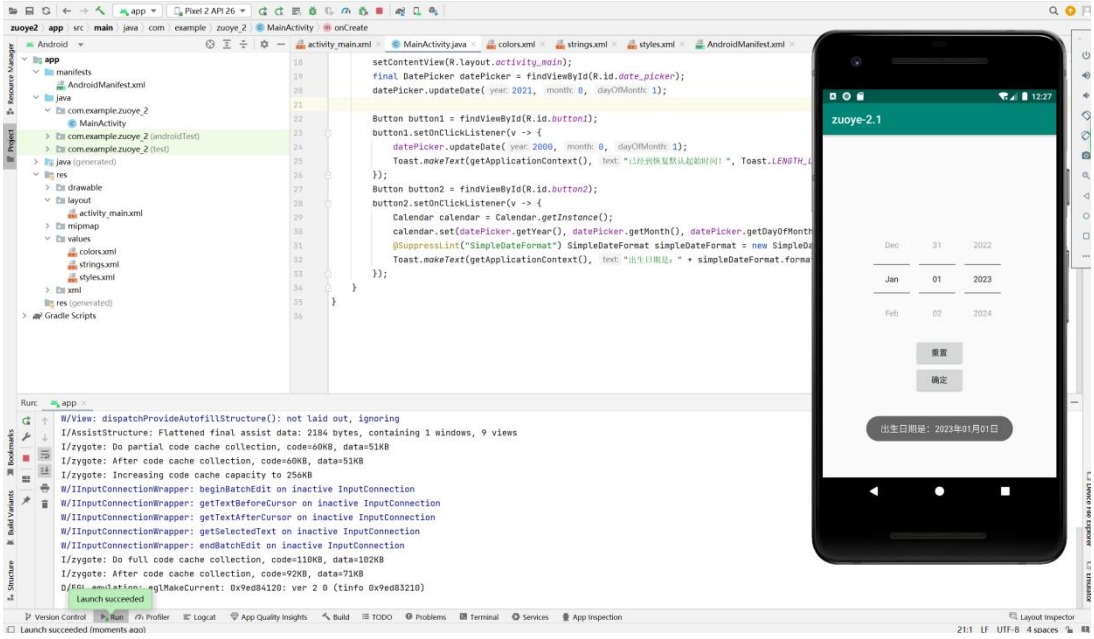
p73　2.3　实现选择出生日期的界面效果。

代码：

```
protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        final DatePicker datePicker = findViewById(R.id.date_picker);
        datePicker.updateDate(2021, 0, 1);
        Button button1 = findViewById(R.id.button1);
```

```
        button1.setOnClickListener(v -> {
            datePicker.updateDate(2000, 0, 1);
            Toast.makeText(getApplicationContext(), "已经到恢复默认起始时间！",
Toast.LENGTH_LONG).show();
        });
        Button button2 = findViewById(R.id.button2);
        button2.setOnClickListener(v -> {
            Calendar calendar = Calendar.getInstance();
            calendar.set(datePicker.getYear(),datePicker.getMonth(), atePicker.getDayOfMonth());
            @SuppressLint("SimpleDateFormat") SimpleDateFormat simpleDateFormat = new
SimpleDateFormat("yyyy 年 MM 月 dd 日");
            Toast.makeText(getApplicationContext(), "出生日期是：" +
simpleDateFormat.format(calendar.getTime()), Toast.LENGTH_SHORT).show();
        });
}
```

运行截图：



第三章：

p87 3.1　实现注册界面的布局效果，至少包括用户名、密码、忘记密码、登录等控件。

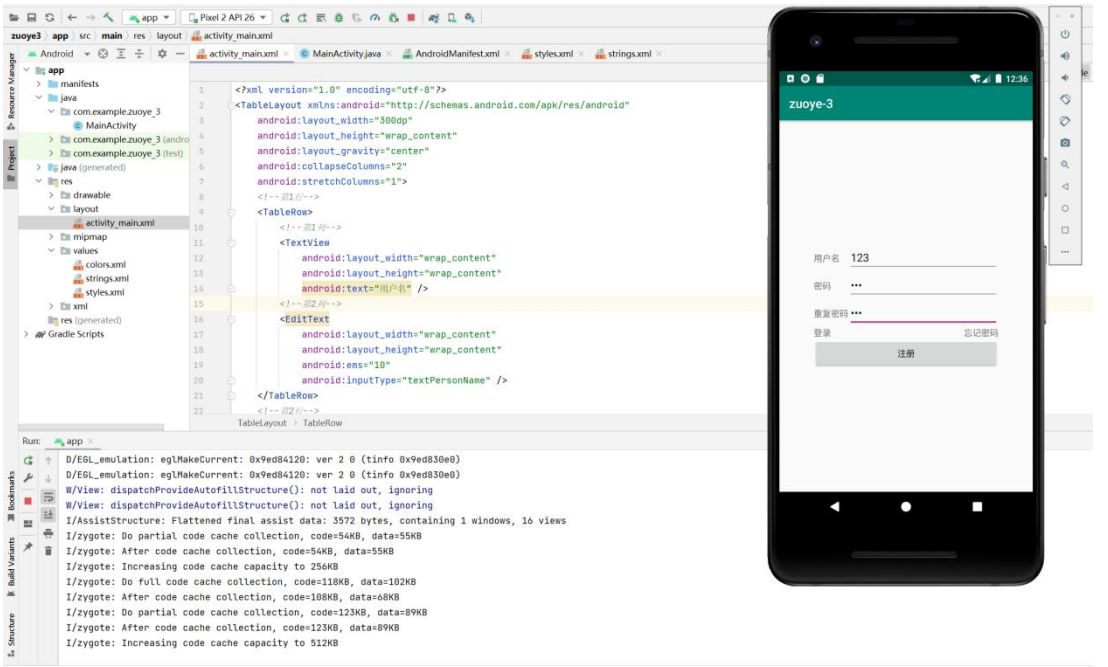
代码：

```
<?xml version="1.0" encoding="utf-8"?>
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="300dp"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:collapseColumns="2"
    android:stretchColumns="1">
    <!--第 1 行-->
    <TableRow>
        <!--第 1 列-->
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="用户名" />
        <!--第 2 列-->
        <EditText
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:ems="10"
            android:inputType="textPersonName" />
    </TableRow>
    <!--第 2 行-->
    <TableRow>
        <!--第 1 列-->
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="密码" />
        <!--第 2 列-->
        <EditText
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:ems="10"
```

```
            android:inputType="textPassword" />
        <!--第 3 列-->
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="忘记密码" />
    </TableRow>
    <TableRow>
        <!--第 1 列-->
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="重复密码" />
        <!--第 2 列-->
        <EditText
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:ems="10"
            android:inputType="textPassword" />
        <!--第 3 列-->
    </TableRow>
    <!--第 3 行-->
    <TableRow>
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="登录" />
        <!--第 2 列-->
        <TextView
            android:id="@+id/textView4"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_column="1"
            android:gravity="end"
            android:text="忘记密码" />
    </TableRow>
    <!--第 4 行-->
    <TableRow>
        <!--第 1 列和第 2 列合并-->
```

```
        <Button
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_span="2"
            android:text="注册" />
    </TableRow>
</TableLayout>
```

运行截图：



第四章：

p122 4.3 实现京东首页的轮播广告效果，至少包括 3 个产品广告的图像。

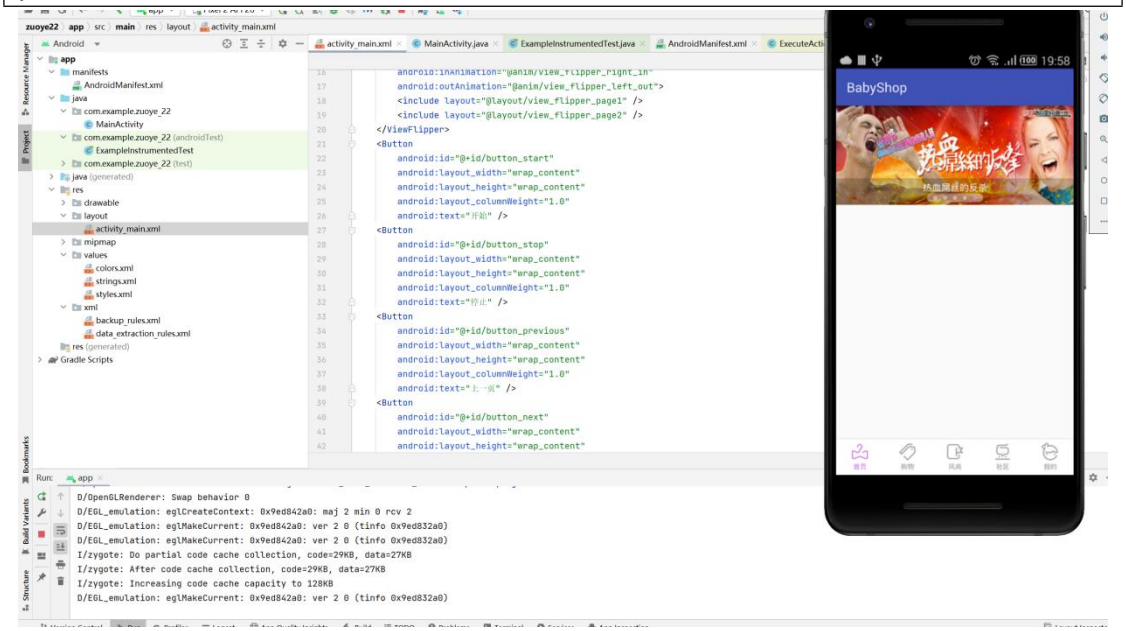
```
<?xml version="1.0" encoding="utf-8"?>
<GridLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="top|fill"
    android:alignmentMode="alignMargins"
    android:background="#eeeeee"
    android:columnCount="4"
    android:rowCount="2">
    <ViewFlipper
```

```
        android:id="@+id/view_flipper"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_columnSpan="4"
        android:flipInterval="3000"
        android:inAnimation="@anim/view_flipper_right_in"
        android:outAnimation="@anim/view_flipper_left_out">
        <include layout="@layout/view_flipper_page1" />
        <include layout="@layout/view_flipper_page2" />
    </ViewFlipper>
    <Button
        android:id="@+id/button_start"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_columnWeight="1.0"
        android:text="开始" />
    <Button
        android:id="@+id/button_stop"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_columnWeight="1.0"
        android:text="停止" />
    <Button
        android:id="@+id/button_previous"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_columnWeight="1.0"
        android:text="上一页" />
    <Button
        android:id="@+id/button_next"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_columnWeight="1.0"
        android:text="下一页" />
</GridLayout>
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android">
    <translate
        android:duration="2000"
```

```
            android:fromXDelta="100%p"
            android:toXDelta="0" />
</set>
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android">
    <translate
            android:duration="2000"
            android:fromXDelta="100%p"
            android:toXDelta="0" />
</set>
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical">
    <ImageView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:adjustViewBounds="true"
            app:srcCompat="@mipmap/img_1656" />
</LinearLayout>
package com.vt.（/  *#/）;

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.Toast;
import android.widget.ViewFlipper;

public class MainActivity extends AppCompatActivity {
private ViewFlipper mVewFlipper;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_main);
            View view3 = View.inflate(MainActivity.this, R.layout.view_flipper_page3, null);
```

```
        ImageView imageView4 = new ImageView(this);
        imageView4.setAdjustViewBounds(true);
        imageView4.setImageResource(R.mipmap.img_9546);
        mVewFlipper = findViewById(R.id.view_flipper);
        mVewFlipper.addView(imageView4);
        mVewFlipper.addView(view3,2);
        mVewFlipper.startFlipping();
        mVewFlipper.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Toast.makeText(MainActivity.this, "当前显示的子视图索引号为"+mVewFlipper.getDisplayedChild(), Toast.LENGTH_SHORT).show();
            }
        });
        Button startBtn = findViewById(R.id.button_start);
        startBtn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                mVewFlipper.startFlipping();
            }
        });
        Button stopBtn = findViewById(R.id.button_stop);
        stopBtn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                mVewFlipper.stopFlipping();
            }
        });
        Button previousBtn = findViewById(R.id.button_previous);
        previousBtn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                mVewFlipper.showPrevious();
            }
        });
        Button nextBtn = findViewById(R.id.button_next);
        nextBtn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
```

```
                mVewFlipper.showNext();
            }
        });
    }
}
```



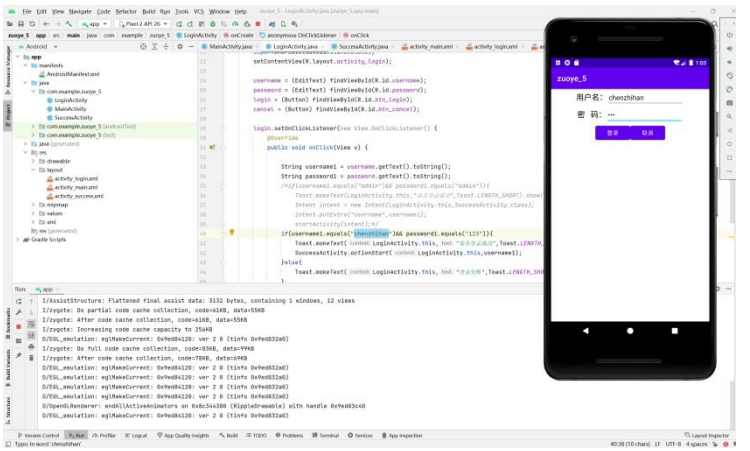
第五章：

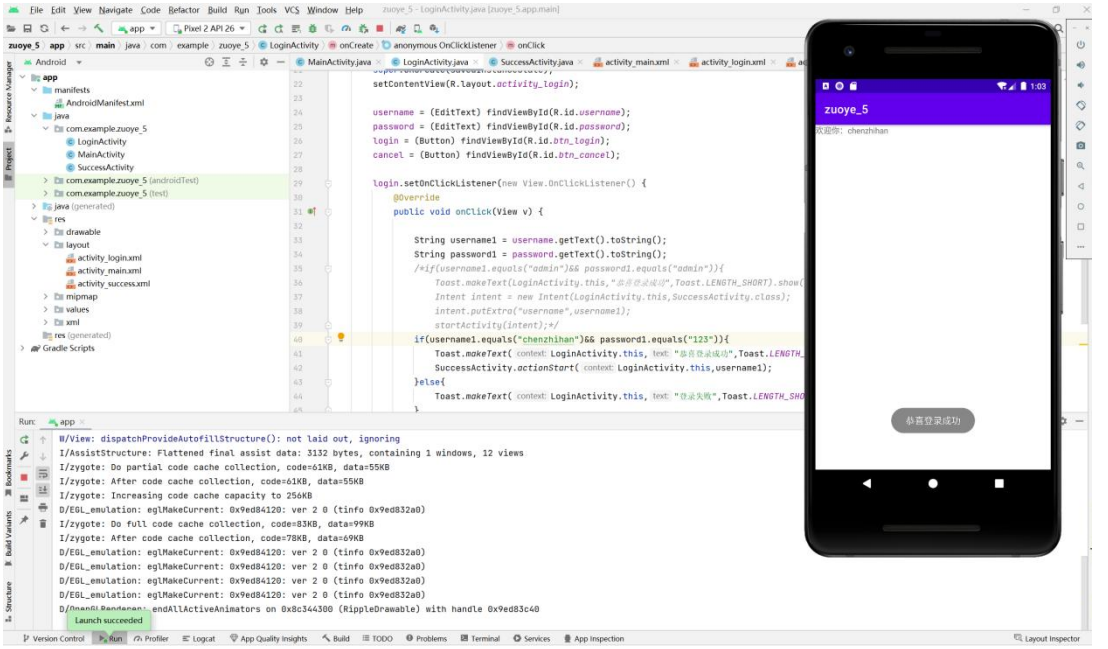
p150 5.1 实现登录后显示用户名的效果。输入用户名、密码后登录，启动一个新的 Activity 显示用户名。

```
public class LoginActivity extends AppCompatActivity {
    private EditText username;
    private EditText password;
    private Button    login;
    private Button cancel;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login);
        username = (EditText) findViewById(R.id.username);
        password = (EditText) findViewById(R.id.password);
        login = (Button) findViewById(R.id.btn_login);
        cancel = (Button) findViewById(R.id.btn_cancel);
        login.setOnClickListener(new View.OnClickListener() {
```

```
                @Override
                public void onClick(View v) {
                    String username1 = username.getText().toString();
                    String password1 = password.getText().toString();
                    /*if(username1.equals("admin")&& password1.equals("admin")){
                        Toast.makeText(LoginActivity.this," 恭 喜 登 录 成 功 ",Toast.LENGTH_SHORT).show();
                        Intent intent = new Intent(LoginActivity.this,SuccessActivity.class);
                        intent.putExtra("username",username1);
                        startActivity(intent);*/
                    if(username1.equals("chenzhihan")&& password1.equals("123")){
                        Toast.makeText(LoginActivity.this," 恭 喜 登 录 成 功 ",Toast.LENGTH_SHORT).show();
                        SuccessActivity.actionStart(LoginActivity.this,username1);
                    }else{
                        Toast.makeText(LoginActivity.this," 登 录 失 败 ",Toast.LENGTH_SHORT).show();
                    }
                }
            });
            cancel.setOnClickListener(new View.OnClickListener(){
                @Override
                public void onClick(View v) {
                    finish();
                }
            });
        }
}
```

代码截图：





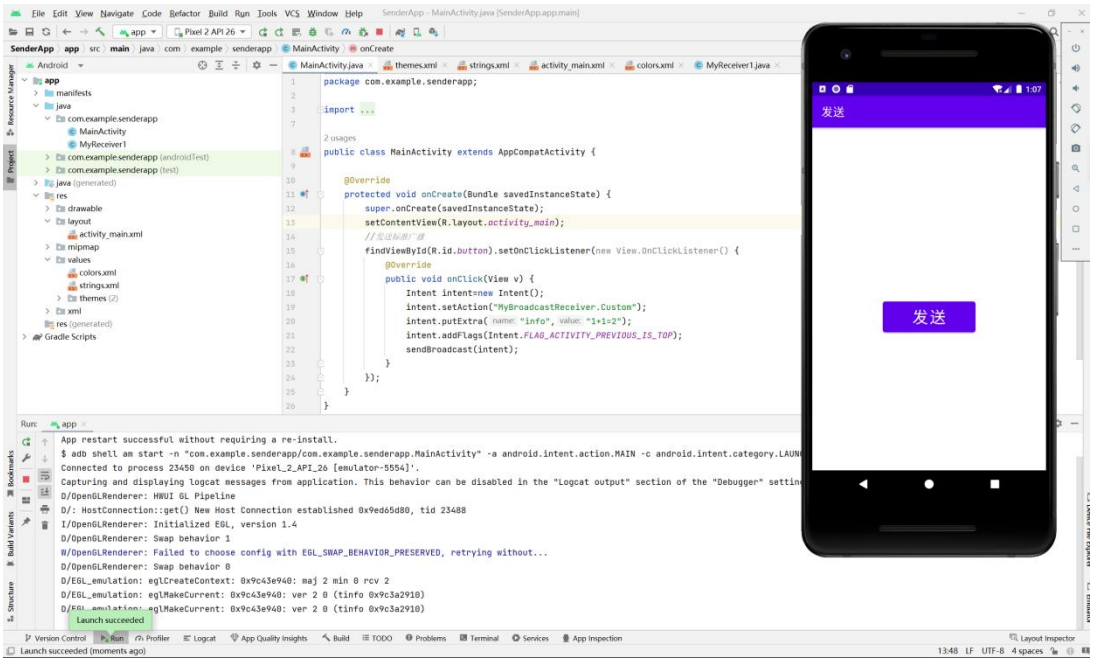
第六章：

p176 6.3 通过广播接收器实现两个 App 之间通过自定义广播进行数据通信。

发送主要代码：

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        //发送标准广播
        findViewById(R.id.button).setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
```

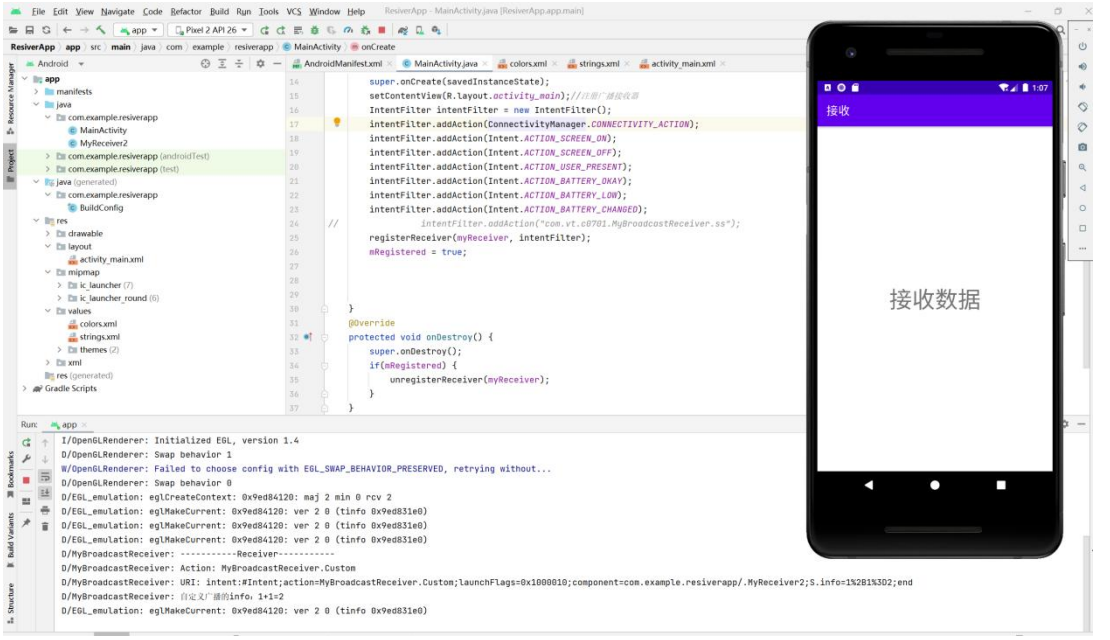
```
                Intent intent=new Intent();
                intent.setAction("MyBroadcastReceiver.Custom");
                intent.putExtra("info","1+1=2");
                intent.addFlags(Intent.FLAG_ACTIVITY_PREVIOUS_IS_TOP);
                sendBroadcast(intent);
            }
        });
    }
}
```



接受主要代码：

```
public class MainActivity extends AppCompatActivity {
    private MyReceiver2 myReceiver = new MyReceiver2();
    private boolean mRegistered = false;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);//注册广播接收器
        IntentFilter intentFilter = new IntentFilter();
        intentFilter.addAction(ConnectivityManager.CONNECTIVITY_ACTION);
        intentFilter.addAction(Intent.ACTION_SCREEN_ON);
        intentFilter.addAction(Intent.ACTION_SCREEN_OFF);
```

```
            intentFilter.addAction(Intent.ACTION_USER_PRESENT);
            intentFilter.addAction(Intent.ACTION_BATTERY_OKAY);
            intentFilter.addAction(Intent.ACTION_BATTERY_LOW);
            intentFilter.addAction(Intent.ACTION_BATTERY_CHANGED);
//          intentFilter.addAction("com.vt.c0701.MyBroadcastReceiver.ss");
            registerReceiver(myReceiver, intentFilter);
            mRegistered = true;
        }
        @Override
        protected void onDestroy() {
            super.onDestroy();
            if(mRegistered) {
                unregisterReceiver(myReceiver);
            }
        }
}
```



第七章：

p213 7.2 使用 SQLite 实现记录登录时间的功能，并且能够查询登录的历史记录。

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <TextView
            android:id="@+id/search_history_item_tv"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_toLeftOf="@id/search_history_item_img"
            android:paddingBottom="10dp"
            android:paddingLeft="10dp"
            android:paddingRight="4dp"
            android:paddingTop="10dp"
            android:textColor="#333333"
            android:textSize="17sp" />

        <ImageView
            android:id="@+id/search_history_item_img"
            android:layout_width="20dp"
            android:layout_height="20dp"
            android:layout_alignParentRight="true"
            android:layout_centerVertical="true"
            android:layout_marginRight="10dp"
            android:src="@drawable/clear" />

    </RelativeLayout>

    <View
        android:layout_width="match_parent"
        android:layout_height="1dp"
        android:layout_marginLeft="10dp"
        android:layout_marginRight="10dp"
        android:background="#e2dbdb" />
</LinearLayout>
```

```
public class HistorySearchAdapter extends RecyclerView.Adapter<HistorySearchAdapter.ViewHolder>
        implements View.OnClickListener {

    private Context mContext;

    private OnItemClickListener mOnItemClickListener;//item 点击监听接口

    private List<String> histotyList = new ArrayList<String>();

    public HistorySearchAdapter(Context context, List<String> histotyList) {
        this.mContext = context;
        this.histotyList = histotyList;
    }

    class ViewHolder extends RecyclerView.ViewHolder {

        private TextView nameTv;
        private ImageView deleteImg;
        private View itemView;

        public ViewHolder(View itemView) {
            super(itemView);
            nameTv = (TextView) itemView.findViewById(R.id.search_history_item_tv);
            deleteImg = (ImageView) itemView.findViewById(R.id.search_history_item_img);
            this.itemView = itemView;
        }
    }

    @Override
    public HistorySearchAdapter.ViewHolder onCreateViewHolder(ViewGroup parent, int viewType) {

        View view = LayoutInflater.from(mContext)
                .inflate(R.layout.item_cjhhistory_search, null);
        ViewHolder holder = new ViewHolder(view);
        return holder;
```

```
    }

    @Override
    public void onBindViewHolder(HistorySearchAdapter.ViewHolder holder, int position) {
        holder.nameTv.setText(histotyList.get(position));
        holder.nameTv.setTag(histotyList.get(position));
        holder.deleteImg.setTag(histotyList.get(position));
        holder.nameTv.setOnClickListener(this);
        holder.deleteImg.setOnClickListener(this);
    }

    public void onClick(View v) {
        switch (v.getId()) {
            case R.id.search_history_item_tv://点击历史纪录名称时调用
                if (mOnItemClickListener != null) {
                    mOnItemClickListener.onItemNameTvClick(v, (String) v.getTag());
                }
                break;
            case R.id.search_history_item_img://点击删除按钮时调用
                if (mOnItemClickListener != null) {
                    mOnItemClickListener.onItemDeleteImgClick(v, (String) v.getTag());
                }
                break;
            default:
        }
    }

    /**
     * 设置 item 点击监听器
     * @param listener
     */
    public void setOnItemClickListener(OnItemClickListener listener) {
        this.mOnItemClickListener = listener;
    }

    @Override
    public int getItemCount() {
        return histotyList.size();
    }
```

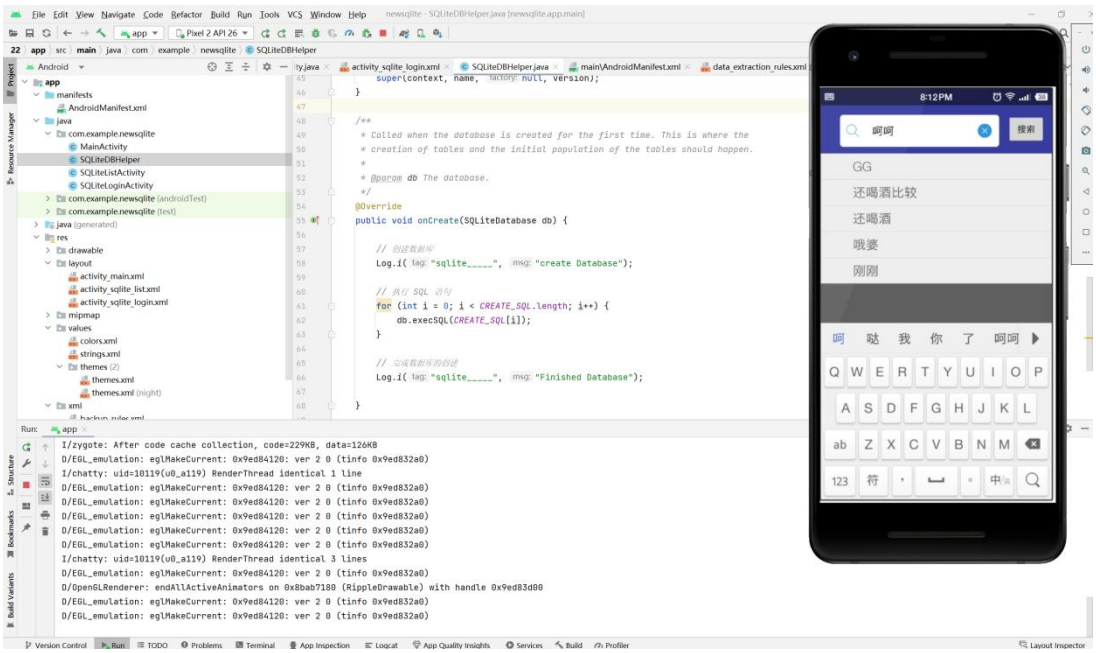
```
    /**
     * item 点击接口
     */
    public interface OnItemClickListener {
        void onItemNameTvClick(View v, String name);//点击历史纪录名称时
        void onItemDeleteImgClick(View v, String name);//点击删除按钮时
}
public class HistoryUtil {

    /**
     * 建表语句
     */
    private static final String CREATE_HISTORY_SEARCH = "create table stock (" +
            "id integer primary key autoincrement, " +
            "name text)";

    public Context mContext;

    private static HistoryUtil mHistorySearchUtil;

    private MyDatabaseHelper mMyDatabaseHelper;

    private HistoryUtil(Context context) {
        mMyDatabaseHelper = new MyDatabaseHelper(context, "demo.db", null, 1);
        this.mContext = context;
    }

    public static HistoryUtil getInstance(Context context) {//得到一个实例
        if (mHistorySearchUtil == null) {
            mHistorySearchUtil = new HistoryUtil(context);
        } else if ((!mHistorySearchUtil.mContext.getClass()
                .equals(context.getClass()))) {判断两个 context 是否相同
            mHistorySearchUtil = new HistoryUtil(context);
        }
        return mHistorySearchUtil;
    }

    /**
```

```
     * 添加一条新纪录
     *
     * @param name
     */
    public void putNewSearch(String name,String tableName) {
        SQLiteDatabase db = mMyDatabaseHelper.getWritableDatabase();
        if (!isExist(name,tableName)) {//判断新纪录是否存在，不存在则添加
            ContentValues values = new ContentValues();
            values.put("name", name);
            db.insert(tableName, null, values);
        }
    }

    /**
     * 判断记录是否存在
     *
     * @param name
     * @return
     */
    public boolean isExist(String name,String tableName) {
        SQLiteDatabase db = mMyDatabaseHelper.getWritableDatabase();
        Cursor cursor = db.rawQuery("select * from " + tableName + " where name=?",
                new String[]{name});
        if (cursor.moveToFirst()) {//如果存在
            return true;
        } else {
            return false;
        }
    }

    /**
     * 查询所有历史纪录
     *
     * @return
     */
    public List<String> queryHistorySearchList(String tableName) {
        SQLiteDatabase db = mMyDatabaseHelper.getWritableDatabase();
        List<String> list = new ArrayList<String>();
        Cursor cursor = db.query(tableName, null, null, null, null, null, null);
```

```
        if (cursor.moveToFirst()) {
            do {
                String name = cursor.getString(cursor.getColumnIndex("name"));
                list.add(name);
            } while (cursor.moveToNext());
        }
        return list;
    }

    /**
     * 删除单条记录
     *
     * @param name
     */
    public void deleteHistorySearch(String name,String tableName) {
        SQLiteDatabase db = mMyDatabaseHelper.getWritableDatabase();
        if (isExist(name,tableName)) {
            db.delete(tableName, "name = " + "'" + name + "'", null);
        }
    }



    /**
     * 删除所有记录
     */
    public void deleteAllHistorySearch(String tableName) {
        SQLiteDatabase db = mMyDatabaseHelper.getWritableDatabase();
        db.delete(tableName, null, null);
    }

    public class MyDatabaseHelper extends SQLiteOpenHelper {

        public MyDatabaseHelper(Context context, String name,
                                SQLiteDatabase.CursorFactory factoty, int version) {
            super(context, name, factoty, version);
        }

        @Override
```

```
        public void onCreate(SQLiteDatabase db) {
            db.execSQL(CREATE_HISTORY_SEARCH);//建表

        }

        @Override
        public void onUpgrade(SQLiteDatabase sqLiteDatabase, int i, int i1) {

        }
    }
}
  <android.support.v4.widget.NestedScrollView
        android:id="@+id/cjh_history_scrollview"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <LinearLayout
            android:id="@+id/history_search_layout"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical">

            <TextView
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:paddingBottom="8dp"
                android:paddingLeft="8dp"
                android:paddingTop="8dp"
                android:text="历史纪录"
                android:textColor="#dddddd"
                android:textSize="13sp" />

            <View
                android:layout_width="match_parent"
                android:layout_height="1dp"
                android:background="#dddddd" />

            <android.support.v7.widget.RecyclerView
                android:id="@+id/history_search_recycler"
```

```
                android:layout_width="match_parent"
                android:layout_height="wrap_content">

            </android.support.v7.widget.RecyclerView>

            <TextView
                android:id="@+id/history_empty_tv"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:gravity="center"
                android:paddingBottom="8dp"
                android:paddingTop="8dp"
                android:text="清除所有历史记录"
                android:textColor="#d6cece"
                android:textSize="13sp" />

        </LinearLayout>
    </android.support.v4.widget.NestedScrollView>
     HistoryUtil.getInstance(this)
                        .putNewSearch(et_ss.getText().toString(), "stock");//保存记录到数据
库
                            getHistoryList();
                            adapter.notifyDataSetChanged();
                            showViews();
```



第八章：

p271 8.1 使用 Camera2 实现定时拍照的功能。

manifest 文中加入：

<uses-permission android:name="android.permission.CAMERA" />

<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />

<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />

关键代码：

```
Timer timer;
private static final String TAG = "MainActivity";
private static final int MAX_PREVIEW_WIDTH = 1080;//预览的最大宽度限制
private static final int MAX_PREVIEW_HEIGHT = 1920;//预览的最大高度限制
private final Context mContext = this;
private Size mPreviewSize;//预览尺寸
private PreviewTextureView mPreviewView;//预览摄像头画面的控件
private Button mButton;//拍照按钮
private Semaphore mSemaphore = new Semaphore(1);//设置信号许可数量
private HandlerThread mBackgroundThread;//后台线程
private Handler mBackgroundHandler;//后台线程的句柄
private String mCameraId = "0";//摄像头 id
private CameraManager mCameraManager;//摄像头管理器
```

```
private CameraCaptureSession mCaptureSession;//摄像头捕捉会话
private CameraDevice mCameraDevice;//摄像头设备
private CaptureRequest.Builder mCaptureRequestBuilder;//捕捉图像请求构建器
private CaptureRequest mCaptureRequest;//捕捉图像请求
private ImageReader mImageReader;//捕获图片的读取器
private File mPhotoFile;//保存图片的文件
private static final String[] PERMISSIONS = {
        Manifest.permission.CAMERA,
        Manifest.permission.READ_EXTERNAL_STORAGE,
        Manifest.permission.WRITE_EXTERNAL_STORAGE
};
// TextureView 的生命周期事件
private TextureView.SurfaceTextureListener mSurfaceTextureListener = new
TextureView.SurfaceTextureListener() {
    @Override
    public void onSurfaceTextureAvailable(SurfaceTexture texture, int width, int height) {
        Log.d(TAG, "TextureView.onSurfaceTextureAvailable()");
        openCamera(width, height);
    }
    @Override
    public void onSurfaceTextureSizeChanged(SurfaceTexture texture, int width, int height)
{ }
    @Override
    public void onSurfaceTextureUpdated(SurfaceTexture texture) { }
    @Override
    public boolean onSurfaceTextureDestroyed(SurfaceTexture texture) {
        return true;
    }
};
// 摄像头设备状态的回调
private CameraDevice.StateCallback mCameraDeviceStateCallback = new
CameraDevice.StateCallback() {
    @Override
    public void onOpened(@NonNull CameraDevice cameraDevice) {
        Log.d(TAG, "CameraDevice.onOpened()");
        mCameraDevice = cameraDevice;
        startPreview();
        mSemaphore.release();//释放 1 个信号许可
    }
```

```
    @Override
    public void onDisconnected(@NonNull CameraDevice cameraDevice) {
        Log.d(TAG, "CameraDevice.onDisconnected()");
        mSemaphore.release();//释放 1 个信号许可
        cameraDevice.close();
        mCameraDevice = null;
    }
    @Override
    public void onError(@NonNull CameraDevice cameraDevice, int error) {
        Log.d(TAG, "CameraDevice.onError()");
        mSemaphore.release();//释放 1 个信号许可
        cameraDevice.close();
        mCameraDevice = null;
    }
};
// ImageReader 的监听器
private ImageReader.OnImageAvailableListener mOnImageAvailableListener = new
ImageReader.OnImageAvailableListener() {
    @Override
    public void onImageAvailable(final ImageReader reader) {
        Log.d(TAG, "ImageReader.onImageAvailable()");
        // 解除 mImageReader 的监听器
        mImageReader.setOnImageAvailableListener(null, null);
        // ImageReader 读取到图像后使用新线程处理图像数据
        ((Activity) mContext).runOnUiThread(new Runnable() {
            @Override
            public void run() {
                handlePhoto(reader);
            }
        });
    }
};
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.test8_1);
    // 初始化
    this.setTitle("Camera2 定时拍摄");
    mPreviewView = findViewById(R.id.texture_view);
```

```
        //获取控件
        TextView tv=findViewById(R.id.tv);
        mButton = findViewById(R.id.button);
        mButton.setOnClickListener(this);
    }
    @Override
    public void onResume() {
        super.onResume();
        startBackgroundThread();
        // 预览显示时打开摄像头，否则通过监听器监测打开摄像头。
        if (mPreviewView.isAvailable()) {
            openCamera(mPreviewView.getWidth(), mPreviewView.getHeight());
        } else {
            mPreviewView.setSurfaceTextureListener(mSurfaceTextureListener);
        }
    }
    @Override
    public void onPause() {
        closeCamera();
        stopBackgroundThread();
        super.onPause();
    }
    //开启后台线程
    private void startBackgroundThread() {
        Log.d(TAG, "startBackgroundThread()");
        mBackgroundThread = new HandlerThread("CameraBackground");
        mBackgroundThread.start();
        mBackgroundHandler = new Handler(mBackgroundThread.getLooper());
    }
    //停止后台线程
    private void stopBackgroundThread() {
        Log.d(TAG, "stopBackgroundThread()");
        mBackgroundThread.quitSafely();
        try {
            mBackgroundThread.join();
            mBackgroundThread = null;
            mBackgroundHandler = null;
        } catch (InterruptedException e) {
            e.printStackTrace();
```

```
        }
    }
    //点击事件
    @Override
    public void onClick(View view) {
        TextView tv=findViewById(R.id.tv);
        mButton.setEnabled(false);
        //定义一个 Handler
        Handler handler=new Handler(){
            @Override
            public void handleMessage(@NonNull Message msg) {
                super.handleMessage(msg);
                if (msg.what>0){
                    tv.setText("倒计时"+msg.what);
                }else {
                    takePhoto();
                    tv.setText("拍摄成功！正在保存");
                    timer.cancel(); //关闭定时器
                }
            }
        };
        timer=new Timer();
        timer.schedule(new TimerTask() {
            int i=10;
            @Override
            public void run() {
                //定义一个消息传过去
                Message msg=new Message();
                msg.what=i--;
                handler.sendMessage(msg);
            }
        },0,1000); //延时 0 毫秒开始计时，每隔 1 秒计时一次
    }
    // 打开摄像头
    @SuppressLint("MissingPermission")
    private void openCamera(int width, int height) {
        Log.d(TAG, "openCamera()");
        if (!Permissions.hasPermissionsGranted(mContext, PERMISSIONS)) {
            Permissions.requestPermissions(mContext, PERMISSIONS);
```

```
            return;
        }
        try {
            // 2500 毫秒内请求获取 1 个许可，否则抛出异常。
            if (!mSemaphore.tryAcquire(2500, TimeUnit.MILLISECONDS)) {
                throw new RuntimeException("打开摄像头超时");
            }
            mCameraManager = (CameraManager) getSystemService(Context.CAMERA_SERVICE);
            // 获取摄像头支持的属性特征
            CameraCharacteristics characteristics = mCameraManager.getCameraCharacteristics(mCameraId);
            // 获取摄像头支持的可用流配置，包括每种格式、大小组合的最小帧持续时间和停顿持续时间。
            StreamConfigurationMap map = characteristics.get(CameraCharacteristics.SCALER_STREAM_CONFIGURATION_MAP);
            // 获取摄像头捕捉的最大尺寸读取图片
            Size largest = Collections.max(Arrays.asList(map.getOutputSizes(ImageFormat.JPEG)), new Util.CompareSizesByArea());
            mImageReader = ImageReader.newInstance(largest.getWidth(), largest.getHeight(), ImageFormat.JPEG, 2);
            // 选择最佳预览尺寸
            mPreviewSize = Util.chooseOptimalSize(mContext, map.getOutputSizes(SurfaceTexture.class), width, height, MAX_PREVIEW_WIDTH, MAX_PREVIEW_HEIGHT, largest);
            // 设置预览尺寸
            mPreviewView.setAspectRatio(mPreviewSize.getHeight(), mPreviewSize.getWidth());
            //打开摄像头
            mCameraManager.openCamera(mCameraId, mCameraDeviceStateCallback, mBackgroundHandler);
        } catch (CameraAccessException e) {
            e.printStackTrace();
        } catch (InterruptedException e) {
            throw new RuntimeException("打开摄像头被中断", e);
        }
    }
    // 创建摄像头预览会话
```

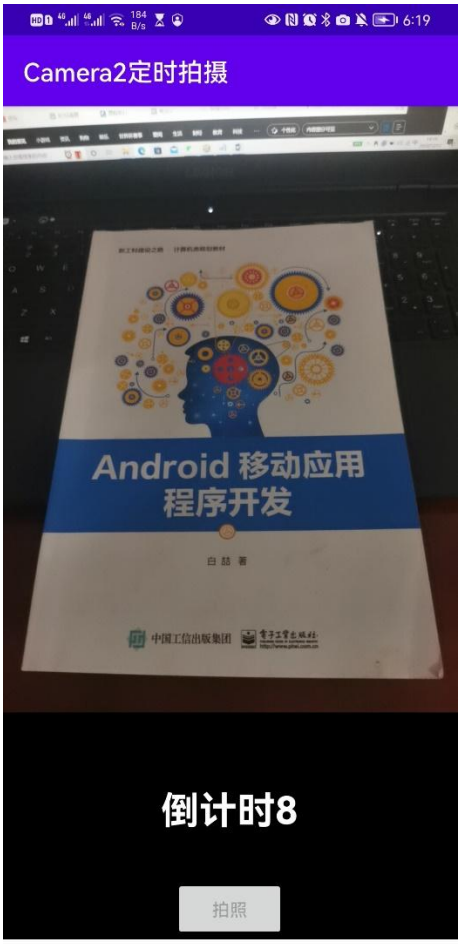
```
private void startPreview() {
    Log.d(TAG, "startPreview()");
    if (null == mCameraDevice || !mPreviewView.isAvailable() || null == mPreviewSize) { return; }
    try {
        closePreview();
        // 设置预览的缓冲区大小
        SurfaceTexture texture = mPreviewView.getSurfaceTexture();
        texture.setDefaultBufferSize(mPreviewSize.getWidth(), mPreviewSize.getHeight());
        // 设置预览输出的 Surface
        Surface surface = new Surface(texture);
        mCaptureRequestBuilder = mCameraDevice.createCaptureRequest(CameraDevice.TEMPLATE_PREVIEW);
        mCaptureRequestBuilder.addTarget(surface);
        // 创建摄像头的捕获会话
        mCameraDevice.createCaptureSession(Arrays.asList(surface, mImageReader.getSurface()), new CameraCaptureSession.StateCallback() {
            @Override
            public void onConfigured(CameraCaptureSession cameraCaptureSession) {
                mCaptureSession = cameraCaptureSession;
                updatePreview();
            }
            @Override
            public void onConfigureFailed(CameraCaptureSession cameraCaptureSession) {
                Toast.makeText(mContext, "摄像头配置失败", Toast.LENGTH_LONG).show();
            }
        }, mBackgroundHandler);
    } catch (CameraAccessException e) {
        e.printStackTrace();
    }
}
// 拍照
private void takePhoto() {
    Log.d(TAG, "takePhoto()");
    if (null == mCameraDevice || !mPreviewView.isAvailable() || null == mPreviewSize) {
        return;
    }
    try {
```

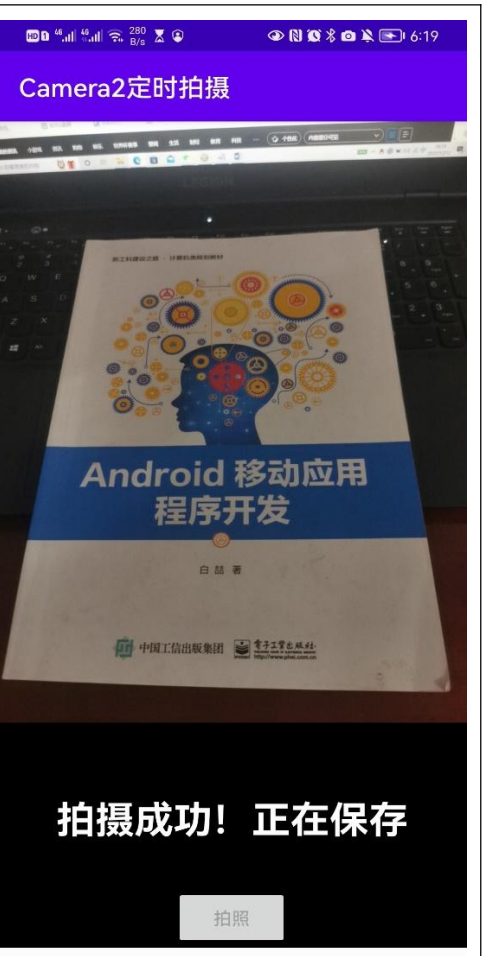
```
        closePreview();
        mCaptureRequestBuilder = mCameraDevice.createCaptureRequest(CameraDevice.TEMPLATE_STILL_CAPTURE);
        // 设置预览的缓冲区大小
        SurfaceTexture texture = mPreviewView.getSurfaceTexture();
        texture.setDefaultBufferSize(mPreviewSize.getWidth(), mPreviewSize.getHeight());
        // 设置预览的 Surface
        List<Surface> surfaces = new ArrayList<>();
        Surface previewSurface = new Surface(texture);
        surfaces.add(previewSurface);
        mCaptureRequestBuilder.addTarget(previewSurface);
        // 设置拍照的 Surface
        Surface imageReaderSurface = mImageReader.getSurface();
        surfaces.add(imageReaderSurface);
        mCaptureRequestBuilder.addTarget(imageReaderSurface);
        // 创建捕捉会话
        mCameraDevice.createCaptureSession(surfaces, new CameraCaptureSession.StateCallback() {
            @Override
            public void onConfigured(@NonNull CameraCaptureSession cameraCaptureSession) {
                Log.d(TAG, "CameraCaptureSession.onConfigured()");
                mCaptureSession = cameraCaptureSession;
                // 添加 ImageReader 监听器处理拍摄照片
                mImageReader.setOnImageAvailableListener(mOnImageAvailableListener, mBackgroundHandler);
                mCaptureRequest = mCaptureRequestBuilder.build();
                try {// 拍摄照片
                    mCaptureSession.capture(mCaptureRequest, null, null);
                } catch (CameraAccessException e) {
                    e.printStackTrace();
                }
            }
            @Override
            public void onConfigureFailed(CameraCaptureSession cameraCaptureSession) {
                Toast.makeText(mContext, "摄像头配置失败", Toast.LENGTH_LONG).show();
            }
        }, mBackgroundHandler);
```

```
        } catch (CameraAccessException e) {
            e.printStackTrace();
        }
    }
    // 处理拍摄的照片
    private void handlePhoto(ImageReader reader){
        Log.d(TAG, "handlePhoto()");
        byte[] photoByte = Util.imageToBytes(reader.acquireNextImage());
        mPhotoFile = Util.creatFile(mContext.getExternalMediaDirs()[0].getAbsolutePath(),
mContext.getResources().getString(R.string.app_name), "jpg");
        // 保存拍摄的照片
        photoByte = Util.saveImage(photoByte, mPhotoFile, 90);
        // 将拍摄的照片显示在系统相册内
        Util.showInAlbum(mContext, mPhotoFile.getAbsolutePath());

        if (mPreviewView.isAvailable()) {
            openCamera(mPreviewView.getWidth(), mPreviewView.getHeight());
        } else {
            mPreviewView.setSurfaceTextureListener(mSurfaceTextureListener);
        }
        TextView tv=findViewById(R.id.tv);
        tv.setText("");
        mButton.setEnabled(true);


    //请求权限回调
    @Override
    public void onRequestPermissionsResult(int requestCode, String[] permissions, int[]
grantResults) {
        Log.d(TAG, "onRequestPermissionsResult()");
        switch (requestCode) {
            case Permissions.REQUEST_PERMISSIONS:
                if (!Permissions.hasPermissionsGranted(mContext, PERMISSIONS)) {
                    Permissions.requestPermissions(mContext, PERMISSIONS);
                }
                break;
        }
    }
```

```
public class Permissions {
    // 权限请求码
    public static final int REQUEST_PERMISSIONS = 1;
    // 是否已经获取权限
    public static boolean hasPermissionsGranted(Context context, String[] permissions) {
        for (String permission : permissions) {
            // 只要有一个拒绝的申请权限就返回 false
            if (ActivityCompat.checkSelfPermission(context, permission) != PackageManager.PERMISSION_GRANTED) {
                return false;
            }
        }
        return true;
    }
    // 请求权限
    public static void requestPermissions(final Context context, String[] permissions) {
        // 如果已经拒绝了权限申请，弹出对话框提示手动开启权限，否则显示权限请求的系统对话框。
        if (shouldShowRequestPermissionRationale(context, permissions)) {
            String msg = "";
            for (int i = 0; i < permissions.length; i++) {
                if (i == 0) {
                    msg = permissions[i];
                } else {
                    msg = msg + "\n" + permissions[i];
                }
            }
            // 自定义对话框
            AlertDialog.Builder adBuilder = new AlertDialog.Builder(context);
            adBuilder.setIcon(R.mipmap.ic_launcher);
            adBuilder.setTitle("需要手动开启以下权限");
            adBuilder.setMessage(msg);
            // 单击确认按钮事件
            adBuilder.setPositiveButton("打开应用设置", new DialogInterface.OnClickListener() {
                @Override
                public void onClick(DialogInterface dialog, int which) {
                    // 打开应用设置
```

```
                        Intent intent = new Intent();

intent.setAction("android.settings.APPLICATION_DETAILS_SETTINGS");
                        intent.setData(Uri.fromParts("package",  context.getPackageName(),
null));
                        context.startActivity(intent);
                        dialog.dismiss();
                    }
                });
                adBuilder.show();
            } else {
                // 显示权限请求的系统对话框
                ActivityCompat.requestPermissions((Activity)  context,  permissions,
REQUEST_PERMISSIONS);
            }
        }
        // 获取是否拒绝过权限请求
        public static boolean shouldShowRequestPermissionRationale(Context context, String[]
permissions) {
            for (String permission : permissions) {
                // 如果有拒绝的申请权限返回 true
                if (ActivityCompat.shouldShowRequestPermissionRationale((Activity) context,
permission)) {
                    return true;
                }
            }
            return false;
        }
}
```


Camera2定时拍摄
Android 移动应用
程序开发
白 喆 著
中国工信出版集团
倒计时8
拍照


Camera2定时拍摄
Android 移动应用
程序开发
白 喆 著
中国工信出版集团
拍摄成功！正在保存
拍照


Android 移动应用
程序开发
白 喆 著
中国工信出版集团